

PYTHON PROGRAMMING

What is Python ?

Python is a general-purpose, dynamically typed, high-level, compiled and interpreted, garbage-collected, and purely object-oriented programming language that supports procedural, object-oriented, and functional programming.

Why the Name Python?

There is a fact behind choosing the name Python. Guido van Rossum was reading the script of a popular BBC comedy series "Monty Python's Flying Circus". It was late on-air 1970s.

Van Rossum wanted to select a name which unique, sort, and little-bit mysterious. So he decided to select naming Python after the "Monty Python's Flying Circus" for their newly created programming language.

History of Python

- Python laid its foundation in the late 1980s.
- The implementation of Python was started in December 1989 by **Guido Van Rossum** at CWI in Netherland.
- In February 1991, **Guido Van Rossum** published the code (labeled version 0.9.0) to alt.sources.
- In 1994, Python 1.0 was released with new features like lambda, map, filter, and reduce.
- Python 2.0 added new features such as list comprehensions, garbage collection systems.
- On December 3, 2008, Python 3.0 (also called "Py3K") was released. It was designed to rectify the fundamental flaw of the language.
- *ABC programming language* is said to be the predecessor of Python language, which was capable of Exception Handling and interfacing with the Amoeba Operating System.
- Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python 2 vs python 3

Python 2	Python 3
<pre>>>7 / 5 Results: 1</pre>	<pre>>>7/5 Results: 1.4</pre>
<pre>print "message"</pre>	<pre>print ("Message")</pre>
<pre>Xrange</pre>	<pre>Range</pre>
<pre>Exception alias mentioned with , except NameError, err:</pre>	<pre>Alias mentioned with 'as' except NameError as err:</pre>
<pre>In order to suppress a newline while printing, you had to use a trailing comma in Python 2. print 2, print 4 2 4</pre>	<pre>in Python 3, you have to use the end keyword argument. print(2, end =" ") print(4) 2 4</pre>
<pre>In Python 2, both != and <> used to work perfectly fine as inequality operators.</pre>	<pre>in python 3, the alternative of <> has been completely removed. Basically now there is only one way of doing it. By using the != operator.</pre>
<pre>raw_input(), input both supported, The difference was that, input() was able to read and store any data type and store it as the same type. In order to store every input as a string, you had to use raw_input()</pre>	<pre>Python 3, the raw input() has been removed. There is only the input() method and it parses the user input as string for every data type.</pre>

Python

- Python is a general-purpose language.
- Python is a Scripting Language
- Scripting language are written using high level language constructs, which makes them easy to learn.
- It has wide range of applications from Web development, scientific and mathematical computing to desktop graphical user Interfaces.
- The syntax of the language is clean and length of the code is relatively short.
- **Python is Interactive** – You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python Programming Cycle

- Relatively shorter than other language.
- There is no compile or link step while it is in built step.
- Python interpreter internally does the compilation
- Python program simply imports library at run time and uses the objects.

Features of python

- **Easy to use and Read** - Python's syntax is clear and easy to read, making it an ideal language for both beginners and experienced programmers. This simplicity can lead to faster development and reduce the chances of errors.

Example: `print("hello")`

- **Dynamically Typed** - The data types of variables are determined during run-time. We do not need to specify the data type of a variable during writing codes.

Example : `x=10 print(x)`

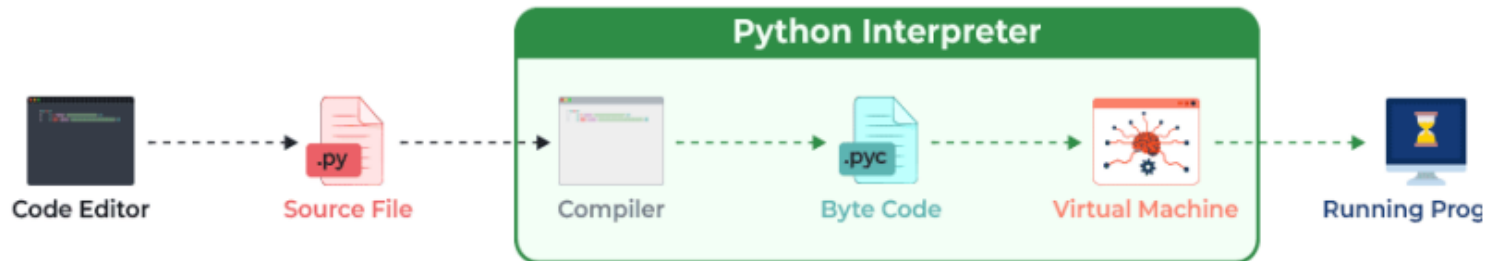
- **High-level** - High-level language means human readable code.
- **Compiled and Interpreted** - Python code first gets compiled into bytecode, and then interpreted line by line. When we download the Python in our system from org we download the default implementation of Python known as CPython. CPython is considered to be Compiled and Interpreted both.

Features of python

- **Garbage Collected** - Memory allocation and deallocation are automatically managed. Programmers do not specifically need to manage the memory.
- **Purely Object-Oriented** - It refers to everything as an object, including numbers and strings.
- **Cross-platform Compatibility** - Python can be easily installed on Windows, macOS, and various Linux distributions, allowing developers to create software that runs across different operating systems.
- **Rich Standard Library** - Python comes with several standard libraries that provide ready-to-use modules and functions for various tasks, ranging from web development and data manipulation to machine learning and networking.
- **Open Source** - Python is an open-source, cost-free programming language. It is utilized in several sectors and disciplines as a result

Internal working of Python

Python directly doesn't convert its code into machine code, something that hardware can understand. It converts it into something called bytecode. So within Python, compilation happens, but it's just not in a machine language. It is into byte code (.pyc or .pyo) and this byte code can't be understood by the CPU. So we need an interpreter called the Python virtual machine to execute the byte codes.



IDEs

Pycharm, Jupyter Notebook, Enthought Canopy, IDLE, Spyder

Difference between Compiler and Interpreter:

Aspect	Compiler	Interpreter
Execution	Translates the entire source code into machine code before execution.	Translates and executes the source code line-by-line or statement-by-statement.
Speed	Generally, it is faster execution after compilation because the code is directly translated to machine code.	Slower execution compared to compiled code, as translation occurs during execution.
Error Detection	Errors are detected and must be corrected after the entire program is compiled.	Errors are detected and must be corrected line-by-line during execution.
Memory Consumption	Typically, it requires more memory initially to store the executable code.	Uses less memory as it directly executes the instructions without creating an executable file.
Development Cycle	Longer development cycle due to the compile-link-execute steps.	Shorter development cycle, as code can be executed directly, making it ideal for rapid testing and debugging.
Portability	The compiled code is platform-specific. Separate compilations are required for different platforms.	Interpreted code is more portable, as the same code can run on any machine with a compatible interpreter.
Examples	C, C++, Rust, and Swift are typically compiled languages.	Python, JavaScript, and Ruby are examples of languages that are commonly interpreted.

First Python Program

- The Python language has many similarities to Perl, C, and Java. However, there are some definite differences between the languages.

```
>>> print ("Hello, Python!" )  
Hello, Python!
```

Add two numbers

```
num1 = 3  
num2 = 5  
sum = num1+num2  
print(sum)
```

Using input() function

- Reads data from user
- Parameter is optional string message/ expression
- Returns the data input as string representation

Using print() function

- The print() function in Python is used to display text or any object to the console or standard output.

Example-

```
name = input("Enter your name: ")  
print("Hello", name)
```

Python Identifiers

- A Python identifier is a name used to identify a variable, function, class, module or other object. An identifier starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9).
- Python does not allow punctuation characters such as @, \$, and % within identifiers.
- Python is a case sensitive programming language.

Python Keywords

Keywords in Python are reserved words that have special meanings and serve specific purposes in the language syntax. Python keywords cannot be used as the names of variables, functions, and classes or any other identifier.

Value Keywords	True, False, None
Operator Keywords	and, or, not, in, is
Control Flow Keywords	if, else, elif, for, while, break, continue, pass, try, except, finally, raise, assert
Function and Class	def, return, lambda, yield, class
Context Management	with, as
Import and Module	import, from, as
Scope and Namespace	global, nonlocal
Async Programming	async, await

Lines and Indentation

- Python provides no braces to indicate blocks of code for class and function definitions or flow control. Blocks of code are denoted by line indentation, which is rigidly enforced.
- The number of spaces in the indentation is variable, but all statements within the block must be indented the same amount

Comments in Python

- A hash sign (#) that is not inside a string literal begins a comment. All characters after the # and up to the end of the physical line are part of the comment and the Python interpreter ignores them.

Quotation in Python

- Python accepts single ('), double (") and triple ('' or ''') quotes to denote string literals, as long as the same type of quote starts and ends the string.
- The triple quotes are used to span the string across multiple lines. For example, all the following are legal –
- `word = 'word' sentence = "This is a sentence."`
`paragraph = """This is a paragraph. It is made up of multiple lines and sentences."""`

Multi-Line Statements

- Statements in Python typically end with a new line. Python does, however, allow the use of the line continuation character (\) to denote that the line should continue. For example –

```
total = item_one + \ item_two + \ item_three
```

- Statements contained within the [], {}, or () brackets do not need to use the line continuation character. For example –

```
days = ['Monday', 'Tuesday', 'Wednesday',  
'Thursday', 'Friday']
```

INTRODUCTION OF PYTHON VARIABLES

- Variable is a name that is used to refer to the memory location
- Python variable is also known as identifier to used to hold the value
- Python variable names can be group of both of letter and digits
- Use lower case letter and upper case letter are different like RAHUL and rahul
- They use the underscore like the variable (_a) & (a_)

Variable Naming Rules

- The first character of variable must be an alphabet or underscore(_)
- All the character except the first character may be an alphabet of lower case like(a-z),upper case like(A-Z) and they use the digits(0-9)
- Identifier name must not use any white space and special character (!,@,#,%)
- Identifier name must not be similar any keyword
- Identifier name are case sensitive for example :- NAME and name are the different name not a same variable
- Example of valid identifier name is :- _a, a123 etc.

Declaring Variables

- Python does not bind us to declare a variable before using the validation
- It allow to create a variable at the required time
- We don't need to declare explicitly variable
- We assign any value to the variable that variable declared
- The equal (=) operator is used to assign the value of variable
- Example :- A=10

A is a variable name and (=) use the assign the value and 10 is a value of A

Example of Python Variables

First Declare variable

Second add next variable

Finally print the variable values

```
A=10
```

```
B=10
```

```
C=A+B
```

```
print(C)
```

```
Name="sourav"
```

```
Age=23
```

```
Salary=21500
```

```
print(Name)
```

```
print(Age)
```

```
print(Salary)
```

Or

```
print(Name, Age, Salary)
```

Multiple Assignment

- Python allows us to assign a value to multiple variable in a single statement It is known as Multiple Assignment
- We can apply multiple assignment in two ways
- Either by assigning a single value to multiple variables or assigning a multiple value in multiple variables

Assigning single value to multiple variables

Example:-

```
a=b=c=100
```

```
print(a)
```

```
print(b)
```

```
print(c)
```

User assign the single value to multiple variable Like a and b and c variables value is 100 and next step is print all the values

Assigning Multiple Values To Multiple Variables

Example:-

```
a,b,c=10,20,30
```

```
print(a)
```

```
print(b)
```

```
print(c)
```

User assign multiple value to multiple variables and second step print all the values

Deleting variables

We can delete the variable use del keyword

Syntax:-

```
del<variable_name>
```

Example :-

```
x=5
```

```
print(x)
```

```
del x
```

```
print(x)    #NameError
```

Assign the value to variable x and print the value

After printing the value Delete the variable x and then print it

Python Operators

- The operator can be defined as a symbol which is responsible for a particular operation between two operands.
- Operators are the pillars of a program on which the logic is built in a specific programming language.

Operators

1. Arithmetic operators
2. Comparison operators
3. Assignment Operators
4. Logical Operators
5. Bitwise Operators
6. Membership Operators
7. Identity Operators

- Arithmetic
- Comparison
- Assignment
- Logical
- Bitwise
- Membership
- Identity
- Operators will be covered in the class on need basis

Operators

+	-	*	//	/	%	**
---	---	---	----	---	---	----

==	!=	>	<	>=	<=
----	----	---	---	----	----

=	+=	-=	*=	//=	/=	%=	**=
---	----	----	----	-----	----	----	-----

and	or	not
-----	----	-----

&		^	~	>>	<<
---	--	---	---	----	----

in	not in
----	--------

is	is not
----	--------

Arithmetic Operators

- Arithmetic operators are used to perform arithmetic operations between two operands.
- It includes + (addition), - (subtraction), *(multiplication), /(divide), %(remainder), //(floor division), and exponent (**) operators.
- Example of Arithmetic operators:-
 - `a=20`
 - `b=10`
 - `c=a+b`
 - `Print(c)`

PYTHON OPERATORS

Operator	Syntax	Description
Addition (+)	x+y	To add two operands.
Subtraction (-)	x-y	To subtract the right operand from the left operand.
Multiplication (*)	x*y	To multiply one operand with the other.
Division (/)	x/y	To divide the left operand by the right operand.
Remainder (%)	x%y	To get the remainder of the division of left operand by the right operand.
Floor Division (//)	x//y	To divide the left operand by the right operand and get the floor value of the quotient.
Exponent (**)	x**y	To get the exponent of the left operand raised to the power of the right operand.

% operator General Concept

General formula of modulus

Mod(a, n) = a - n * floor(a/n)

- $-1 \% 8 = \text{mod}(-1, 8) = -1 - 8 * \text{floor}(-1/8) = -1 - 8 * -1 = -1 + 8 = 7$

Add denominator to numerator till we get 0 or positive number

- $1 \% -8 = \text{mod}(1, -8) = 1 - (-8) * \text{floor}(1/-8) = 1 + 8 * -1 = 1 - 8 = -7$

Add denominator to numerator till we get 0 or negative number

- $-1 \% -8 = \text{mod}(-1, -8) = -1 - (-8) * \text{floor}(-1/-8) = -1 + 8 * 0 = -1 + 0 = -1$

Result is negative of normal result if not 0

- $1 \% 8 = \text{mod}(1, 8) = 1 - (8) * \text{floor}(1/8) = 1 - 8 * 0 = 1 + 0 = 1$

Normal case

```
--  
>>> -1%8  
7  
>>> 1%-8  
-7  
>>> -1%-8  
-1  
>>> 1%8  
1  
>>>
```

Comparison operator

- The name itself is explaining that this operator is used to compare different things or values with one another.
- In Python, the Comparison operator is used to analyze either side of the values and decides the relation between them.
- Comparison operator is also termed as a relational operator because it explains the connection between them.

Comparison operator

- Comparison operators are used to comparing the value of the two operands and returns Boolean true or false accordingly. Example a=10, b=20

Operator	Description	Example
==	If the values of two operands are equal, then the condition becomes true.	(a == b) is not true.
!=	If values of two operands are not equal, then condition becomes true.	
<>	If values of two operands are not equal, then condition becomes true.	(a <> b) is true. This is similar to != operator.
>	If the value of left operand is greater than the value of right operand, then condition becomes true.	(a > b) is not true.
<	If the value of left operand is less than the value of right operand, then condition becomes true.	(a < b) is true.
>=	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.	(a >= b) is not true.
<=	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.	(a <= b) is true.

Assignment Operators

- The assignment operators are used to assign the value of the right expression to the left operand. For example: $a=10$, $b=20$

Operator	Example	Equivalent Expression ($m=15$)	Result
=	$y = a+b$	$y = 10 + 20$	30
+=	$m += 10$	$m = m + 10$	25
-=	$m -= 10$	$m = m - 10$	5
*=	$m *= 10$	$m = m * 10$	150
/=	$m /= 10$	$m = m / 10$	1.5
%=	$m \% = 10$	$m = m \% 10$	5
**=	$m ** = 2$	$m = m ** 2$ or $m = m^2$	225
//=	$m //= 10$	$m = m // 10$	1

Logical Operators

- In Python, Logical operators are used on conditional statements (either True or False). They perform Logical AND, Logical OR, and Logical NOT operations.

Operator	Description	Syntax	Example
and	Returns True if both the operands are true	x and y	x>7 and x>10
or	Returns True if either of the operands is true	x or y	x<7 or x>15
not	Returns True if the operand is false	not x	not(x>7 and x> 10)

Bitwise Operators

- The bitwise operators perform bit by bit operation on the values of the two operands.

Types of Bitwise Operators

Operator	Name	Example	Result
&	Bitwise AND	6 & 3	2
	Bitwise OR	10 10	10
^	Bitwise XOR	2 ^ 2	0
~	Bitwise 1's complement	~9	-10
<<	Left-Shift	10 << 2	40
>>	Right-Shift	10 >> 2	2

Bitwise Operators

It performs logical AND operation on the integer value after converting an integer to a binary value and gives the result as a decimal value it returns true only if both operands are true otherwise it returns false

Example of Bitwise AND operators:-

a=7

b=4

c=5

Print(a&b)

Print(a&c)

Print(b&c)

Bitwise Operators

- Bitwise or operators:-

It performs logical OR operation on the integer value after converting integer value to binary value and gives the result a decimal value . It returns false only if both operands are true otherwise it returns true.

Example of Bitwise OR operators:-

```
a=7
```

```
b=4
```

```
c=5
```

```
print(a | b)
```

```
print(a | c)
```

```
print(b| c)
```

Bitwise Operators

- Bitwise XOR operators:-

It performs logical XOR operation on the binary value of a integer and gives the result as a decimal value.

Example of a Bitwise XOR operators:-

`a=7`

`b=4`

`c=5`

`print( a ^ c)`

`print(b ^ c)`

Bitwise Operators

- Bitwise 1's complement $\sim$:-
- It performs 1's complements operation it invert each bit of binary value and returns the bitwise negation of a value as a result.

Example of Bitwise 1'st complements:-

`a=7`

`b=4`

`c=3`

`print(~a, ~b, ~c)`

Bitwise Operators

- Bitwise left-shift << operators:-

The left-shift << operators performs a shifting bit of a value by a given number of the place and fills 0's to new positions.

Example of Bitwise left-shift << operators:-

```
print(4 << 2)
```

```
print(5 << 3)
```

Bitwise Operators

- Bitwise right-shift >>

The left-shift >> operator performs shifting a bit of value to the right by a given Number of places.

Example of Bitwise right-shift >> operators:-

```
print(4 >> 2)
```

```
print(5 >> 2)
```

Membership operators

- Membership operators:- python membership operators are used to check for membership of objects in sequence ,
- such as string , list , tuple. It checks whether the given value or variable is present in a given sequence.
- If present it will return true else false.
- Python there are two membership operators IN and NOT IN

Membership operators

- IN operator:- it returns a result as true. If it finds a given object in the sequence. Otherwise it return false.
- Example of IN OPERATORS:-

```
list =[11,15,21,29,50,70]
```

```
num=15
```

```
If num in list:
```

```
    print("number in present")
```

```
else:
```

```
    print("number not in present")
```

Membership operators

- NOT IN OPERATOR:-it returns true if the object is not present in a given sequence.
- Otherwise it return false.
- Example of NOT In OPERATOR:-

Tuple =(11,15,21,29,50,70)

num=15

If num not in tuple:

print("number is present")

else:

print("number is not present")

IDENTITY OPERATORS

- Identity operators check whether the value of two variable is the same or not.
- This operator is known as reference - quality operators.
- Because the identity operators compares value according to two variable memory addresses
- Python has 2 identity operators **is** and **is not**

IDENTITY OPERATORS

- IS OPERATOR:- this operator return Boolean True or False.
- It return true if the memory address first value is equal to the second value otherwise it return false.
- Example of is operators:-

x=10

y=11

z=10

print(x is y)

print(x is z)

IDENTITY OPERATORS

- IS NOT OPERATORS: the is not operators return Boolean value either true or false.
- It return true if the first value is not equal to the second value
- Otherwise it return false.

Example of is not operator:

```
x=10
```

```
y=11
```

```
z=10
```

```
print(x is not y)
```

```
print(x is not z)
```

Precedence and Associativity of Operators in Python

Precedence :

Precedence is the order in which different operators in an expression are evaluated. Operators with higher precedence are executed before those with lower precedence, which ensures that expressions are processed according to the rules of operator priority.

Associativity:

Associativity defines the order in which operators of the same precedence level are evaluated in an expression. It determines whether operations are performed from left to right or right to left.

Operators	Meaning
()	Parentheses
**	Exponent
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, >=, <, <=, is, is not, in, not in	Comparisons, Identity, Membership operators
not	Logical NOT
and	Logical AND
or	Logical OR
= %= /= //= -= += *= **=	Assignment expression

Operators	Associativity
() Highest precedence	Left - Right
**	Right - Left
+x, -x, ~x	Left - Right
*, /, //, %	Left - Right
+, -	Left - Right
<<, >>	Left - Right
&	Left - Right
^	Left - Right
	Left - Right
Is, is not, in, not in, <, <=, >, >=, ==, !=	Left - Right
Not x	Left - Right
And	Left - Right
Or	Left - Right
If else	Left - Right
Lambda	Left - Right
=, +=, -=, *=, /= Lowest Precedence	Right - Left

Example:

1.

```
exp= 100 + 200 / 10 - 3 * 10  
print(exp)
```
2.

```
result = (4 + 6) * (2 ** 3) / 5 - 7 % 3  
print(result)
```
3.

```
result = not (True and False) or (False or True)  
print(result)
```
4.

```
result = 5.0 * 2 + 3.0 ** 2 - 8.0 / 4  
print(result)
```
5.

```
result = 8 * 3 + 2 - 12 // 4 ** 2 // (5 % 3) + 1 - 6 - 2 * 1 + 1 ** 2  
print(result)
```

Answers:

1.90.0

2.15.0

3.True

4.17.0

5.20

Assume, Num1=5, Num2=6.

1. Num1 < Num2
2. Num1 <= Num2
3. Num1 > Num2
4. Num1 >= Num2
5. Num1 != Num2

Identify the expressions which would result in True?

- a. 1, 2, 3
- b. 1, 2, 5
- c. 2, 3, 4
- d. 1, 4, 5

☐ a

☒ b

☐ c

☐ d

What is the output of the below code snippet?

```
num1 = 10  
num2 = 5  
num2 *= num1  
print(num2)
```

This is same as `num2=num2*num1`. Python allows you to do this kind of short circuiting with any arithmetic operator

☐ 50

☐ 10

☐ 25

☐ 100

Q3 of 3

Assume, Num1=5, Num2=6.

1. (Num1 < Num2) and (Num1 != Num2)
2. (Num2 >= Num1) or (Num1 > Num2)
3. not (Num1 == Num2)

Identify the expressions which would result in True?

- a. All the given options
- b. 1, 2
- c. 2, 3
- d. 1, 3

☒ a

☐ b

☐ c

☐ d

Python Objects

- Everything in python is object.
- Every object specifically data has three properties type, id and value
- ID remains same once data is created.
- Objects whose value can change after creation is called mutable and whose value can't be changed is called immutable
- Mutable examples: **list, set, dictionary**
- Immutable examples: **int, float, decimal, bool, string, tuple, and range.**

Python Data Types

- Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data.
- Since everything is an object in Python programming, data types are actually classes and variables are instances (objects) of these classes.

Python Data Types

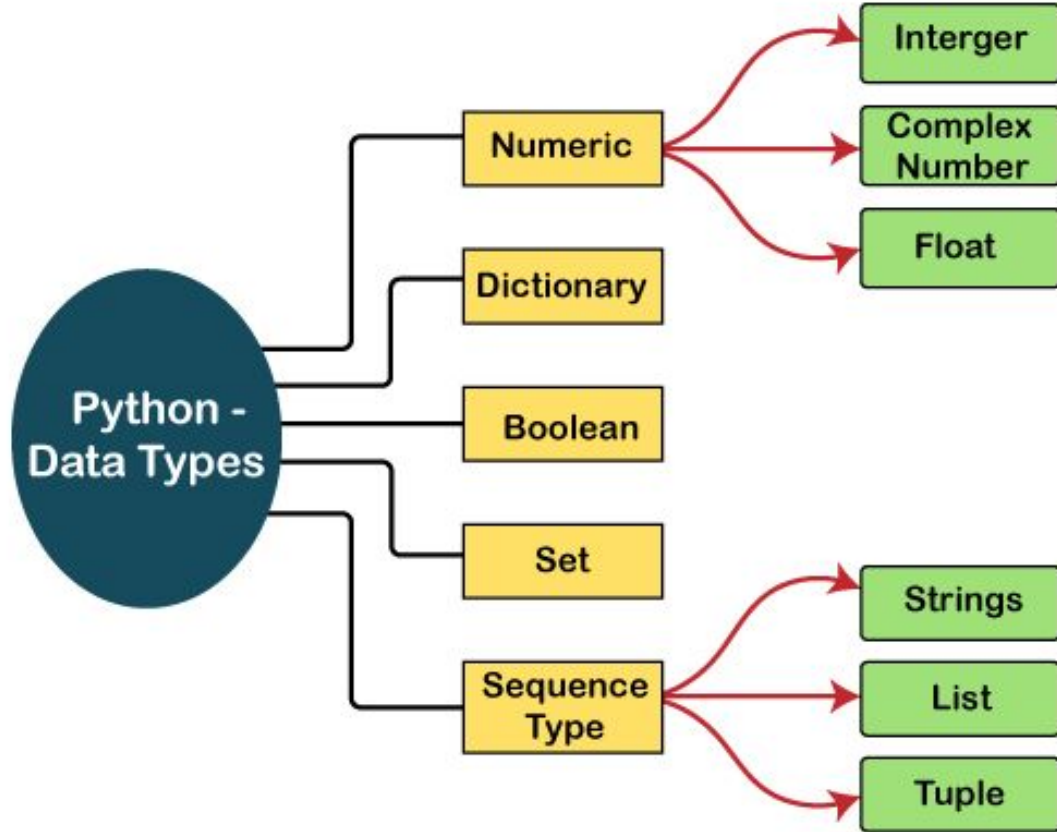
```
a = 5
```

The variable **a** holds integer value five and we did not define its type.

Python interpreter will automatically interpret variables **a** as an integer type.

Python enables us to check the type of the variable used in the program.

Python provides us the **type()** function, which returns the type of the variable passed.



NUMERIC DATA TYPE

Numeric Data Type

- Number stores numeric values.
- The integer, float, and complex values belong to a Python Numbers data-type.
- Python provides the **type()** function to know the data-type of the variable.
- Similarly, the **isinstance()** function is used to check an object belongs to a particular class.

Numbers

Python supports three types of numeric data.

1. **Int** - Integer value can be any length such as integers 10, 2, 29, -20, -150 etc. Python has no restriction on the length of an integer. Its value belongs to **int**
2. **Float** - Float is used to store floating-point numbers like 1.9, 9.902, 15.2, etc. It is accurate upto 15 decimal points.
3. **complex** - A complex number contains an ordered pair, i.e., $x + iy$ where x and y denote the real and imaginary parts, respectively. The complex numbers like 2.14j, $2.0 + 2.3j$, etc.

Numbers

```
a = 5
```

```
print("The type of a", type(a))
```

```
b = 40.5
```

```
print("The type of b", type(b))
```

```
c = 1+3j
```

```
print("The type of c", type(c))
```

```
print(" c is a complex number", isinstance(1+3j,complex))
```

Output

The type of a <class 'int'>

The type of b <class 'float'>

The type of c <class 'complex'>

c is complex number: True

String Data Type

String

Python string is the collection of the characters surrounded by single quotes, double quotes, or triple quotes.

Syntax:

```
str = "Hi Python !"
```

Here, if we check the type of the variable str using a Python script

```
print(type(str)) #then it will print a string (str).
```

In Python, strings are treated as the sequence of characters, which means that Python doesn't support the character data-type;

instead, a single character written as 'p' is treated as the string of length 1.

String

#Using single quotes

```
str1 = 'Hello Python'  
  
print(str1)
```

#Using double quotes

```
str2 = "Hello Python"  
  
print(str2)
```

#Using triple quotes

```
str3 = """Triple quotes are  
generally used for represent the  
multiline or docstring"  
  
print(str3)
```

Strings indexing and splitting

`str = "HELLO"`

H	E	L	L	O
0	1	2	3	4

`str[0] = 'H'`

`str[1] = 'E'`

`str[2] = 'L'`

`str[3] = 'L'`

`str[4] = 'O'`

Strings indexing and splitting

the slice operator `[]` is used to access the individual characters of the string.

However, we can use the `:` (colon) operator in Python to access the substring from the given string.

`str = "HELLO"`

H	E	L	L	O
0	1	2	3	4

`str[0] = 'H'`

`str[:] = 'HELLO'`

`str[1] = 'E'`

`str[0:] = 'HELLO'`

`str[2] = 'L'`

`str[:5] = 'HELLO'`

`str[3] = 'L'`

`str[:3] = 'HEL'`

`str[4] = 'O'`

`str[0:2] = 'HE'`

`str[1:4] = 'ELL'`

Strings indexing and splitting

We can do the negative slicing in the string; it starts from the rightmost character, which is indicated as -1.

The second rightmost index indicates -2, and so on.

str = "HELLO"				
H	E	L	L	O
-5	-4	-3	-2	-1

str[-1] = 'O'

str[-3:-1] = 'LL'

str[-2] = 'L'

str[-4:-1] = 'ELL'

str[-3] = 'L'

str[-5:-3] = 'HE'

str[-4] = 'E'

str[-4:] = 'ELLO'

str[-5] = 'H'

str[::-1] = 'OLLEH'

String Concatenation

```
a = "Hello"
```

```
b = "World"
```

```
c = a + b
```

```
print(c)
```

To add a space between them, add a " ":

```
a = "Hello"
```

```
b = "World"
```

```
c = a + " " + b
```

```
print(c)
```

LIST

Python List

- A list in Python is used to store the sequence of various types of data.
- Python lists are mutable type its mean we can modify its element after it created.
- A list can be defined as a collection of values or items of different types.
- The items in the list are separated with the comma (,) and enclosed with the square brackets [].

A list can be defined as below

1. `L1 = ["John", 102, "USA"]`
2. `L2 = [1, 2, 3, 4, 5, 6]`

List is a collection which is ordered and changeable.

Allows duplicate members.

List indexing and splitting

List = [0, 1, 2, 3, 4, 5]

0	1	2	3	4	5
---	---	---	---	---	---

List[0] = 0

List[0:] = [0,1,2,3,4,5]

List[1] = 1

List[:] = [0,1,2,3,4,5]

List[2] = 2

List[2:4] = [2, 3]

List[3] = 3

List[1:3] = [1, 2]

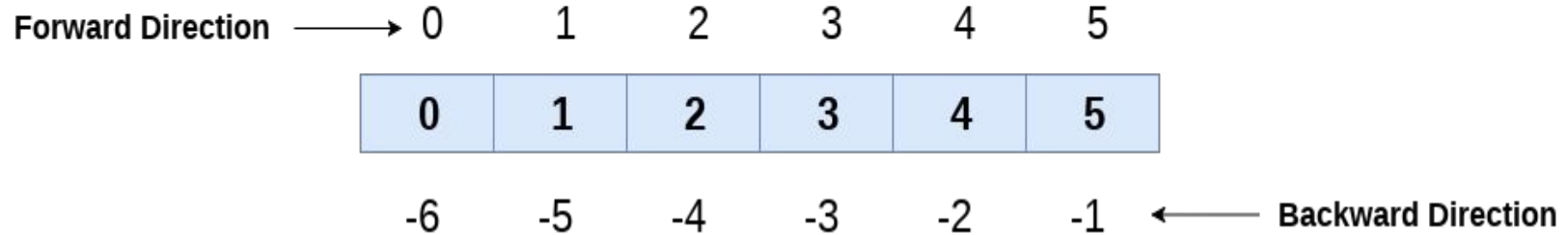
List[4] = 4

List[:4] = [0, 1, 2, 3]

List[5] = 5

List indexing and splitting

List = [0, 1, 2, 3, 4, 5]



Unlike other languages, Python provides the flexibility to use the negative indexing also. The negative indices are counted from the right.

Tuple

- Tuples are used to store multiple items in a single variable.
- A tuple is a collection which is **ordered and unchangeable**.
- Tuples are written with round brackets.

Create a Tuple:

```
thistuple = ("apple", "banana", "cherry")
```

```
print(thistuple)
```

Tuple

- Tuple items are **ordered, unchangeable, and allow duplicate values.**
- Tuple items are indexed, the first item has index [0], the second item has index [1] etc.
- When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.
- Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created. (Immutable)
- **Allow Duplicates** : they can have items with the same value:

Tuple

T1 = (101, "Peter", 22)

T2 = ("Apple", "Banana", "Orange")

T3 = 10,20,30,40,50

T4 = (10,)

T5 = ()

Tuple indexing and slicing

The indexing and slicing in the tuple are similar to lists.

The indexing in the tuple starts from 0 and goes to `length(tuple) - 1`.

The items in the tuple can be accessed by using the index `[]` operator.

Python also allows us to use the colon operator to access multiple items in the tuple.

Tuple indexing and slicing

Tuple = (0, 1, 2, 3, 4, 5)

0	1	2	3	4	5
---	---	---	---	---	---

Tuple[0] = 0 Tuple[0:] = (0, 1, 2, 3, 4, 5)

Tuple[1] = 1 Tuple[:] = (0, 1, 2, 3, 4, 5)

Tuple[2] = 2 Tuple[2:4] = (2, 3)

Tuple[3] = 3 Tuple[1:3] = (1, 2)

Tuple[4] = 4 Tuple[:4] = (0, 1, 2, 3)

Tuple[5] = 5

Boolean

- A **Boolean data type** is a data type that can hold one of two built-in values: True or False. These values represent truth values, where True indicates something is correct or exists, and False indicates the opposite. In Python, Boolean values are represented by the bool class.

Note: True and False must begin with a capital T and F respectively; otherwise, Python will raise a syntax error.

```
[12] a= True
      b= False
      print("type of a ;",type(a))
      print("type of b ;",type(b))
```

⇒ type of a ; <class 'bool'>
type of b ; <class 'bool'>

▶ a=true
print(type(a))

⇒ -----
NameError Traceback (most recent call last)
 <ipython-input-14-85c936bd1a71> in <cell line: 1>()
----> 1 a=true
 2 print(type(a))

NameError: name 'true' is not defined

Set

- Set is an **unordered collection** of data types that is **unindexed,iterable, immutable**, and has **no duplicate** elements.
- The order of elements in a set is undefined though it may consist of various elements.
- Set can be created by placing the sequence inside curly braces, separated by a 'comma'.
- The type of elements in a set need not be the same, various mixed-up data type values can also be passed to the set.
- Sets can also be created by using the built-in set() function

✓
0s [17] #Empty Set
set1={}
print(set1)

⇌ {}

✓
0s  print("Type of set")
print(type(set1))

⇌ Type of set
<class 'set'>

✓
0s [18] #different operations on set
Set1={1,2,3,4,5}
Set2={4,5,6,7,8}
print(Set1|Set2)
print(Set1&Set2)
print(Set1-Set2)
print(Set1^Set2)

⇌ {1, 2, 3, 4, 5, 6, 7, 8}
{4, 5}
{1, 2, 3}
{1, 2, 3, 6, 7, 8}

✓
0s [19] #lenth of set
set1={1,2,3,4,5}
print(len(set1))

⇌ 5

✓
0s [20] #set with diffent types of element
set1={1,'python', 10.75, 'hello'}
print(set1)

⇌ {'python', 1, 10.75, 'hello'}

```
[21] #Adding the elements to the set
      set1[0]='annaconda'
      print(set1)
```



```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-21-9b092b7158d1> in <cell line: 2>()
      1 #Adding the elements to the set
----> 2 set1[0]='annaconda'
      3 print(set1)
```

TypeError: 'set' object does not support item assignment

Next steps: [Explain error](#)

```
[22] #incorrect way to Accessing the elements of the set
      set1={1,2,3,4,5}
      print(set1[0])
```



```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-22-7f12ce6ce343> in <cell line: 3>()
      1 #incorrect way to Accessing the elements of the set
      2 set1={1,2,3,4,5}
----> 3 print(set1[0])
```

TypeError: 'set' object is not subscriptable

Next steps: [Explain error](#)

```
[23] #correct way to accessing the elments of the set
      for i in set1:
          print(i,end=" ")
```



```
1 2 3 4 5
```

✓
Js



```
#defining the set by built in functions
# Creating a Set
set1 = set()
print("Initial blank set")
print(set1)
# Adding element and tuple to the Set
set1.add(8)
set1.add(9)
set1.add((6,7))
print("\nSet after Addition of Three elements")
print(set1)
#
```



Initial blank set
set()

Set after Addition of Three elements
{8, 9, (6, 7)}

Dictionary

- Dictionary in Python is an **unordered collection(till python 3.6 version) , (from python 3.7 onwards Dictionary is ordered)** of data values, used to store data values like a map, which unlike other Data Types that hold only single value as an element, Dictionary holds key:value pair.
- Key-value is provided in the dictionary to make it more optimized. Each key-value pair in a Dictionary is separated by a colon :, whereas each key is separated by a 'comma'.
- Dictionary can be created by placing a sequence of elements within curly {} braces, separated by 'comma'. Values in a dictionary can be of any datatype and can be duplicated, whereas keys can't be repeated and must be immutable.
- Dictionary can also be created by the built-in function dict()

Note:Dictionary keys are case sensitive, same name but different cases of Key will be treated distinctly.

```

# Creating an empty Dictionary
Dict = {}
print("Empty Dictionary: ")
print(Dict)

print("Type of Dict: ")
print(type(Dict))

# Creating a Dictionary
# with Integer Keys
Dict = {1: 'Monty', 2: 'For', 3: 'Python'}
print("\nDictionary with the use of Integer Keys: ")
print(Dict)

# Creating a Dictionary
# with Mixed keys
Dict = {'Name': 'python', 1: [1, 2, 3, 4]}
print("\nDictionary with the use of Mixed Keys: ")
print(Dict)

# Creating a Dictionary
# with dict() method
Dict = dict({1: 'Python', 2: 'java', 3: 'c++'})
print("\nDictionary with the use of dict(): ")
print(Dict)

```

```

> Empty Dictionary:
{}
Type of Dict:
<class 'dict'>

```

```

Dictionary with the use of Integer Keys:
{1: 'Monty', 2: 'For', 3: 'Python'}

```

```

Dictionary with the use of Mixed Keys:
{'Name': 'python', 1: [1, 2, 3, 4]}

```

```

Dictionary with the use of dict():
{1: 'Python', 2: 'java', 3: 'c++'}

```

```
▶ #length of Dictionary
print("length of Dict()")
print(len(Dict))

#Accessing elements of dictionary
print("Accessing elements of dictionary")
print(Dict[1])
print(Dict[2])
print(Dict[3])
```

```
⇒ length of Dict()
3
Accessing elements of dictionary
Python
java
c++
```

```
[10] #Accessing all the elements of dictionary
print("dictionary all items using loop")
for i in Dict.items():
    print(i)
```

```
⇒ dictionary all items using loop
(1, 'Python')
(2, 'java')
(3, 'c++')
```

Type Conversion (Type Cast)

- Conversion of value of one type to other
- We are used to `int` $\longleftrightarrow$ `float` conversion in Math
 - Integer 3 is treated as float 3.0 when a real number is expected
 - Float 3.6 is truncated as 3, or rounded up as 4 for integer contexts
- Type names also work as Type converter functions in Python

Type Conversion Functions:

`int()` – converts any data type into integer type

`float()` – converts any data type into float type

`str()` – Converts any var. value/Constant into string type

`ord()` – converts characters into integer

`hex()` – converts integers to hexadecimal

`oct()` – converts integer to octal

Many other will see in future

Type Conversion Examples

```
In [20]: int(2.5)  
Out[20]: 2
```

```
In [21]: int(2.3)  
Out[21]: 2
```

```
In [22]: int(3.9)  
Out[22]: 3
```

```
In [23]: float(3)  
Out[23]: 3.0
```

```
In [24]: int('73')  
Out[24]: 73
```

```
In [25]: int('Acads')  
Traceback (most recent call last):
```

```
File "<ipython-input-25-90ec37205222>", line 1, in <module>  
    int('Acads')
```

Note that float to int conversion is truncation, not rounding off

Reading binary octal and hex number

```
>>> a=0xff
>>> a
255
>>> b=0o121
>>> b
81
>>> binary=0b111
>>> binary
7
>>> octal=0o121
>>> octal
81
>>> hexa=0xff
>>> hexa
255
>>>
```

Converting Decimal to non decimal

```
>>> #converting decimal to binary
```

```
>>> bin(15)
```

```
'0b1111'
```

```
>>> #converting decimal to octal
```

```
>>> oct(15)
```

```
'0o17'
```

```
>>> #converting decimal to hexadecimal
```

```
>>> hex(15)
```

```
'0xf'
```

```
>>> |
```